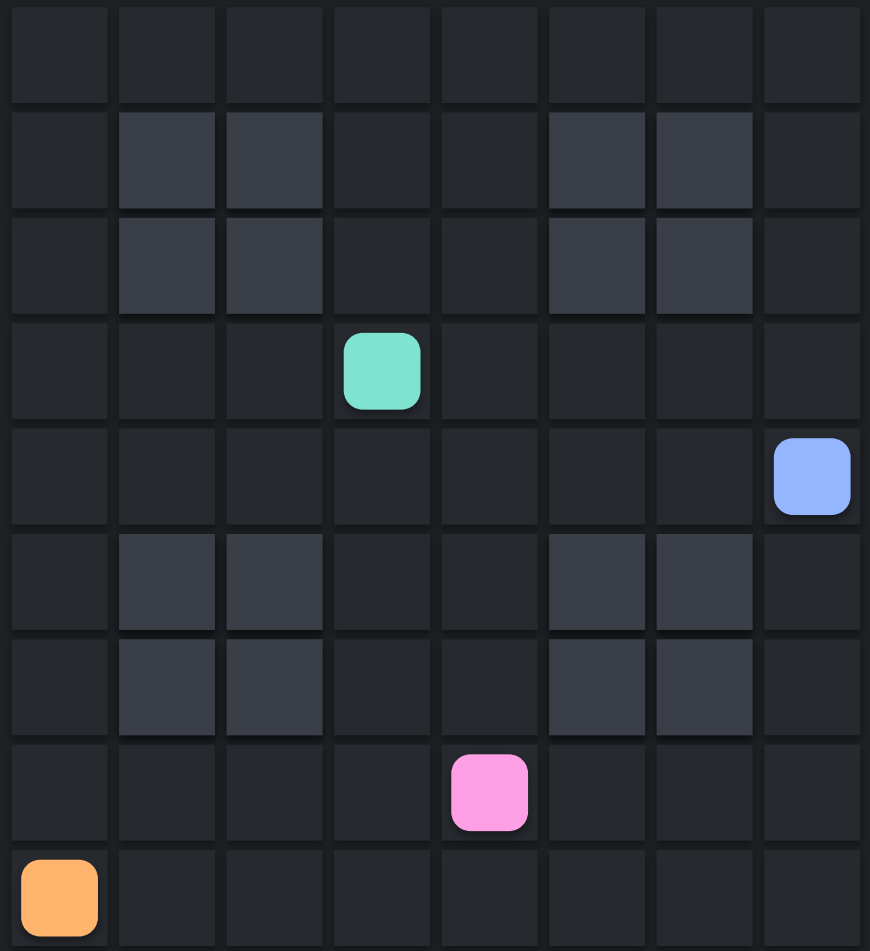
INŻYNIERIA OPROGRAMOWANIA · CHECKPOINT 2

RoboFleet

Stany kluczowych obiektów biznesowych i aktywności kluczowych funkcjonalności



Anna Granos · Mikołaj Zawrotniak · Patrick Fongang · Bartek Dźwigała

Wydział MiNI, Politechnika Warszawska · listopad 2026

ZAKRES DOKUMENTU

Pięć obiektów biznesowych, pięć funkcjonalności

Diagramy stanów

SM-1 **Przebieg symulacji**
SimulationRun.status

SM-2 **Zlecenie transportowe**
TransportOrder.status

SM-3 **Robot w trakcie przebiegu**
RobotAgent.state + podmaszyna Ruch

SM-4 **Magazyn**
Warehouse.status

SM-5 **Konto użytkownika**
User.isActive, role

Diagramy aktywności

AD-1 **Krok symulacji (tick)**
co tickIntervalMs w każdym przebiegu

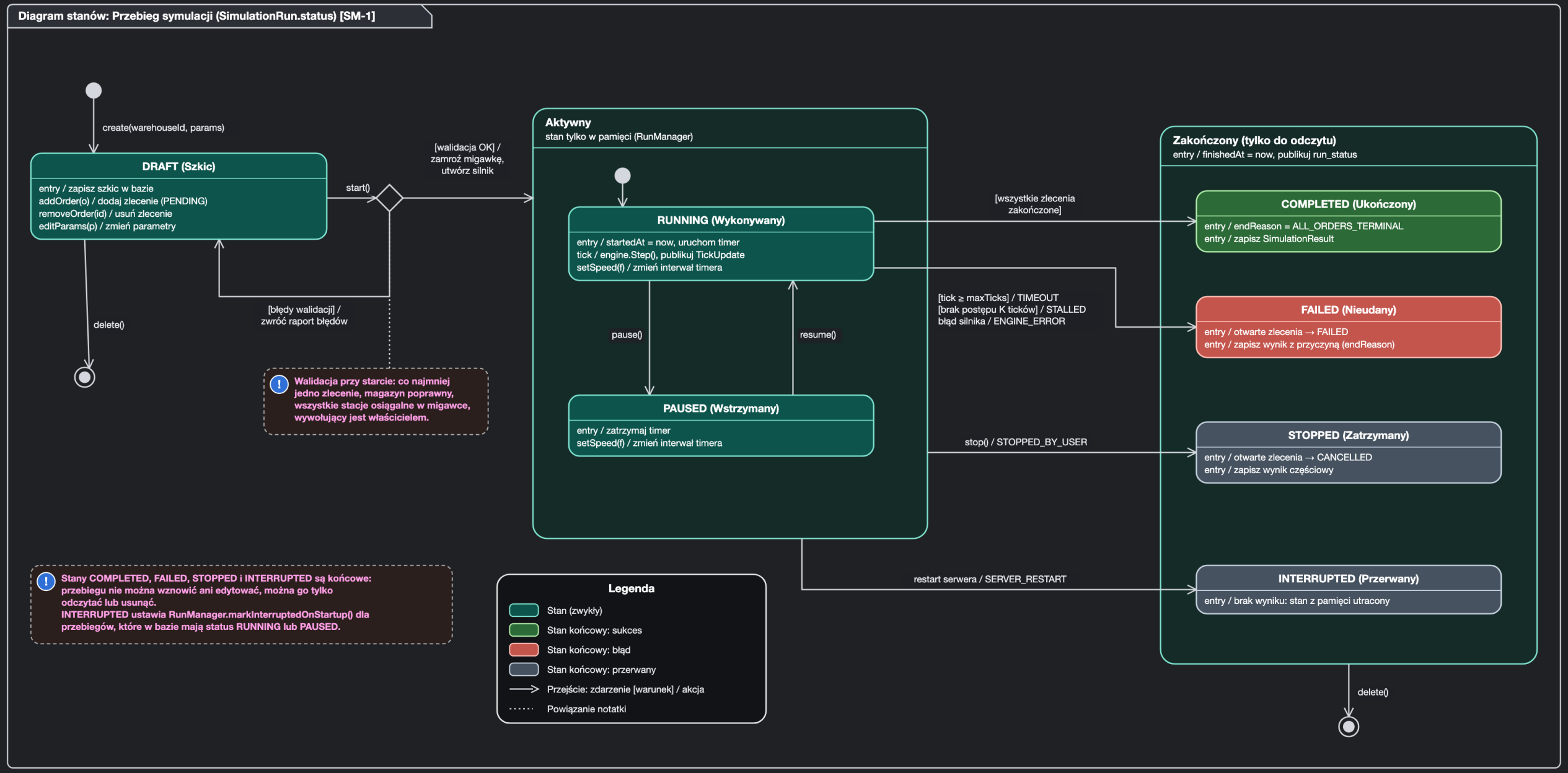
AD-2 **Uruchomienie przebiegu**
walidacja, migawka, start silnika

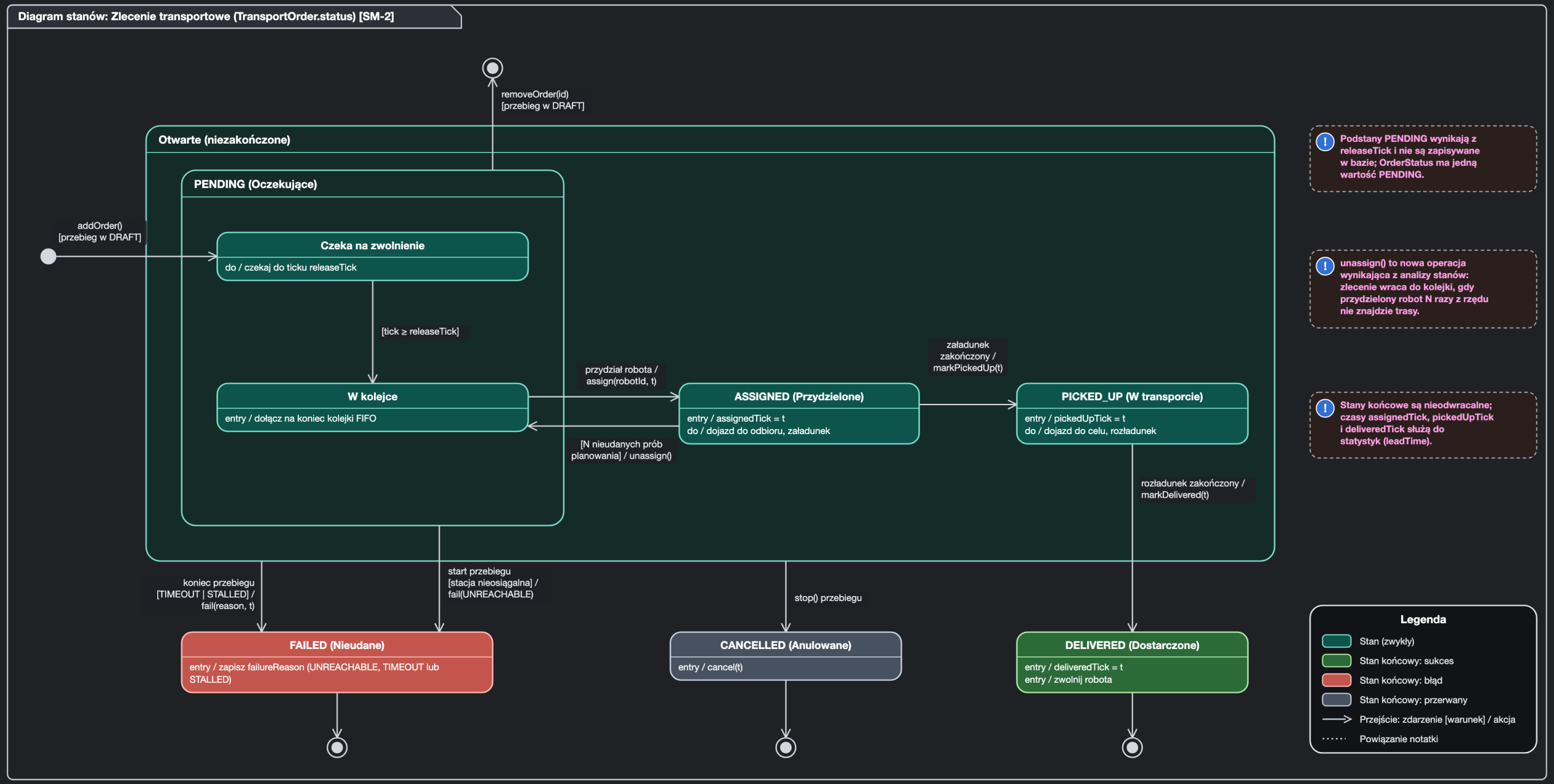
AD-3 **Dodanie zlecenia**
trzy walidacje, pętla formularza

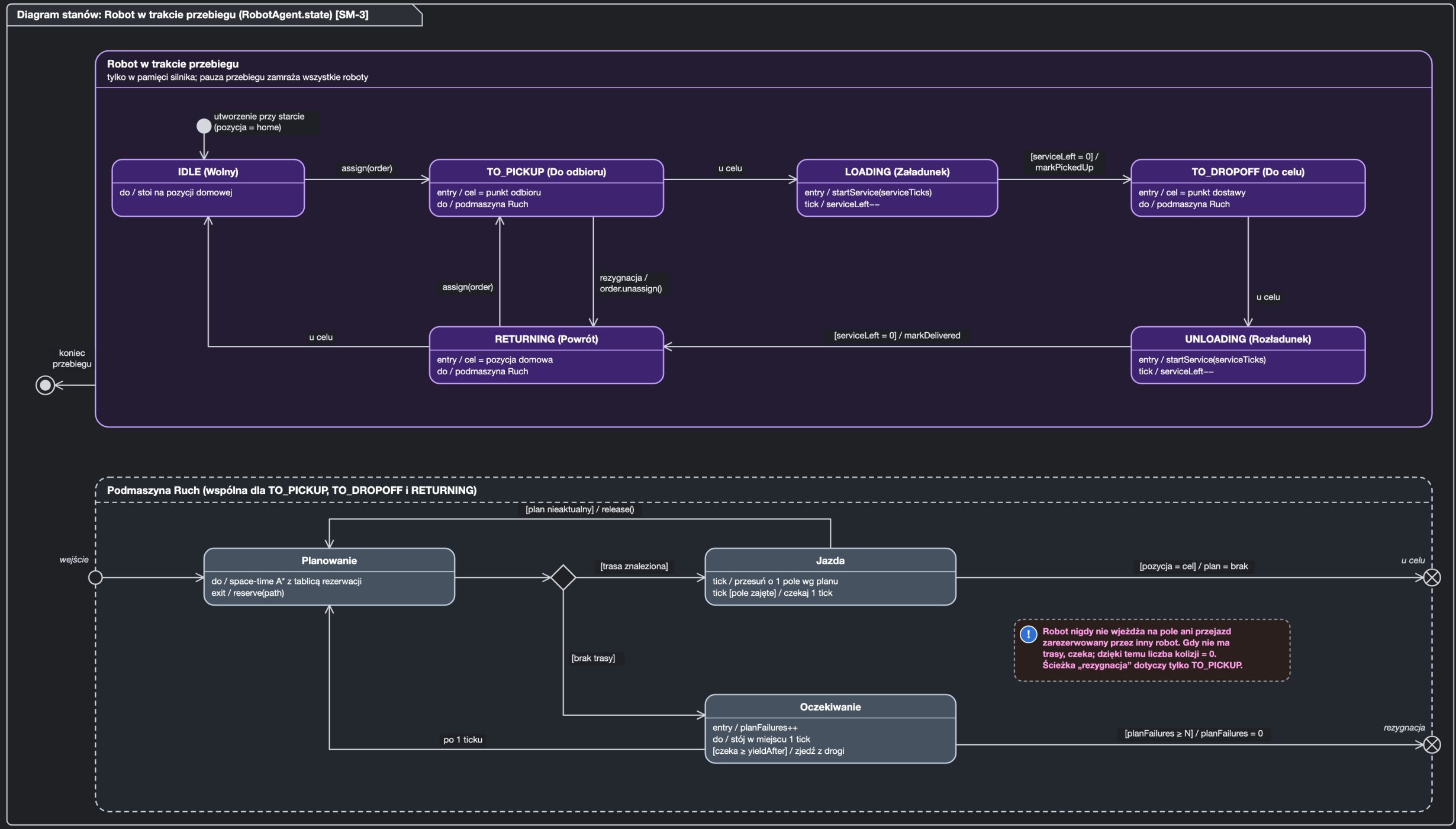
AD-4 **Projektowanie i zapis magazynu**
wersjonowanie, LayoutValidator

AD-5 **Podgląd na żywo i ponowne połączenie**
SignalR, region przerywalny

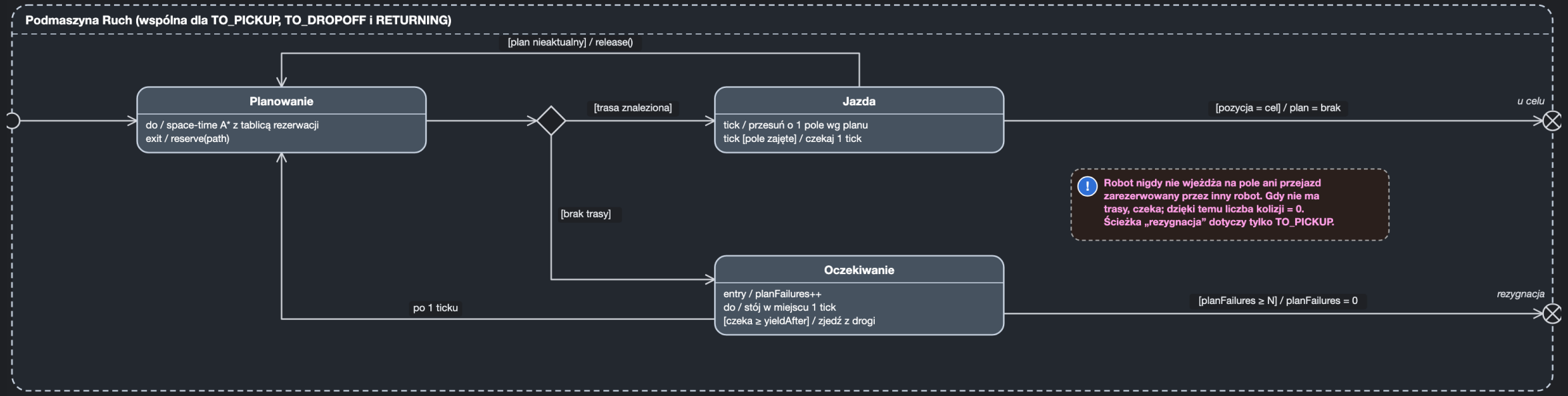
Każdy diagram stanów ma tabelę przejść nietypowych, każdy diagram aktywności tabelę ścieżek alternatywnych. Nazwy klas i enumeracji są zgodne z diagramami klas z Checkpointu 1.


Diagram stanów: Przebieg symulacji (SimulationRun.status) [SM-1]
create(warehouseId, params)
DRAFT (Szkic)
entry / zapisz szkic w bazie
addOrder(o) / dodaj zlecenie (PENDING)
removeOrder(id) / usuń zlecenie
editParams(p) / zmień parametry
start()
[walidacja OK] / zamroź migawkę, utwórz silnik
delete()
[błędy walidacji] / zwróć raport błędów
Walidacja przy starcie: co najmniej jedno zlecenie, magazyn poprawny, wszystkie stacje osiągalne w migawce, wywołujący jest właścicielem.
Aktywny
stan tylko w pamięci (RunManager)
RUNNING (Wykonywany)
entry / startedAt = now, uruchom timer
tick / engine.Step(), publikuj TickUpdate
setSpeed(f) / zmień interwał timera
pause()
resume()
PAUSED (Wstrzymany)
entry / zatrzymaj timer
setSpeed(f) / zmień interwał timera
[wszystkie zlecenia zakończone]
[tick ≥ maxTicks] / TIMEOUT
[brak postępu K ticków] / STALLED
błąd silnika / ENGINE_ERROR
stop() / STOPPED_BY_USER
restart serwera / SERVER_RESTART
Zakończony (tylko do odczytu)
entry / finishedAt = now, publikuj run_status
COMPLETED (Ukończony)
entry / endReason = ALL_ORDERS_TERMINAL
entry / zapisz SimulationResult
FAILED (Nieudany)
entry / otwarte zlecenia → FAILED
entry / zapisz wynik z przyczyną (endReason)
STOPPED (Zatrzymany)
entry / otwarte zlecenia → CANCELLED
entry / zapisz wynik częściowy
INTERRUPTED (Przerwany)
entry / brak wyniku: stan z pamięci utracony
delete()
Stany COMPLETED, FAILED, STOPPED i INTERRUPTED są końcowe: przebiegu nie można wznowić ani edytować, można go tylko odczytać lub usunąć.
INTERRUPTED ustawia RunManager.markInterruptedOnStartup() dla przebiegów, które w bazie mają status RUNNING lub PAUSED.
Legenda
Stan (zwykły)
Stan końcowy: sukces
Stan końcowy: błąd
Stan końcowy: przerwany
Przejście: zdarzenie [warunek] / akcja
Powiązanie notatki


Diagram stanów: Zlecenie transportowe (TransportOrder.status) [SM-2]
removeOrder(id)
[przebieg w DRAFT]
Otwarte (niezakończone)
PENDING (Oczekujące)
addOrder()
[przebieg w DRAFT]
Czeka na zwolnienie
do / czekaj do ticku releaseTick
[tick ≥ releaseTick]
W kolejce
entry / dołącz na koniec kolejki FIFO
przydział robota /
assign(robotId, t)
[N nieudanych prób
planowania] / unassign()
ASSIGNED (Przydzielone)
entry / assignedTick = t
do / dojazd do odbioru, załadunek
załadunek
zakończony /
markPickedUp(t)
PICKED_UP (W transporcie)
entry / pickedUpTick = t
do / dojazd do celu, rozładunek
rozładunek zakończony /
markDelivered(t)
koniec przebiegu
[TIMEOUT | STALLED] /
fail(reason, t)
start przebiegu
[stacja nieosiągalna] /
fail(UNREACHABLE)
stop() przebiegu
FAILED (Nieudane)
entry / zapisz failureReason (UNREACHABLE, TIMEOUT lub
STALLED)
CANCELLED (Anulowane)
entry / cancel(t)
DELIVERED (Dostarczone)
entry / deliveredTick = t
entry / zwolnij robota
Podstany PENDING wynikają z releaseTick i nie są zapisywane w bazie; OrderStatus ma jedną wartość PENDING.
unassign() to nowa operacja wynikająca z analizy stanów: zlecenie wraca do kolejki, gdy przydzielony robot N razy z rzędu nie znajdzie trasy.
Stany końcowe są nieodwracalne; czasy assignedTick, pickedUpTick i deliveredTick służą do statystyk (leadTime).
Legenda
Stan (zwykły)
Stan końcowy: sukces
Stan końcowy: błąd
Stan końcowy: przerwany
Przejście: zdarzenie [warunek] / akcja
Powiązanie notatki


Diagram stanów: Robot w trakcie przebiegu (RobotAgent.state) [SM-3]
Robot w trakcie przebiegu
tylko w pamięci silnika; pauza przebiegu zamraża wszystkie roboty
utworzenie przy starcie
(pozycja = home)
IDLE (Wolny)
do / stoi na pozycji domowej
assign(order)
TO_PICKUP (Do odbioru)
entry / cel = punkt odbioru
do / podmaszyna Ruch
u celu
LOADING (Załadunek)
entry / startService(serviceTicks)
tick / serviceLeft−−
[serviceLeft = 0] /
markPickedUp
TO_DROPOFF (Do celu)
entry / cel = punkt dostawy
do / podmaszyna Ruch
u celu
UNLOADING (Rozładunek)
entry / startService(serviceTicks)
tick / serviceLeft−−
[serviceLeft = 0] / markDelivered
RETURNING (Powrót)
entry / cel = pozycja domowa
do / podmaszyna Ruch
u celu
assign(order)
rezygnacja /
order.unassign()
koniec
przebiegu
Podmaszyna Ruch (wspólna dla TO_PICKUP, TO_DROPOFF i RETURNING)
wejście
Planowanie
do / space-time A* z tablicą rezerwacji
exit / reserve(path)
[plan nieaktualny] / release()
[trasa znaleziona]
Jazda
tick / przesuń o 1 pole wg planu
tick [pole zajęte] / czekaj 1 tick
[pozycja = cel] / plan = brak
u celu
[brak trasy]
Oczekiwanie
entry / planFailures++
do / stój w miejscu 1 tick
[czeka ≥ yieldAfter] / zjedź z drogi
po 1 ticku
[planFailures ≥ N] / planFailures = 0
rezygnacja
Robot nigdy nie wjeżdża na pole ani przejazd
zarezerwowany przez inny robot. Gdy nie ma
trasy, czeka; dzięki temu liczba kolizji = 0.
Ścieżka „rezygnacja" dotyczy tylko TO_PICKUP.

Podmaszyna Ruch: planowanie, jazda, oczekiwanie



Planowanie

space-time A* z tablicą rezerwacji; exit / reserve(path)

Jazda

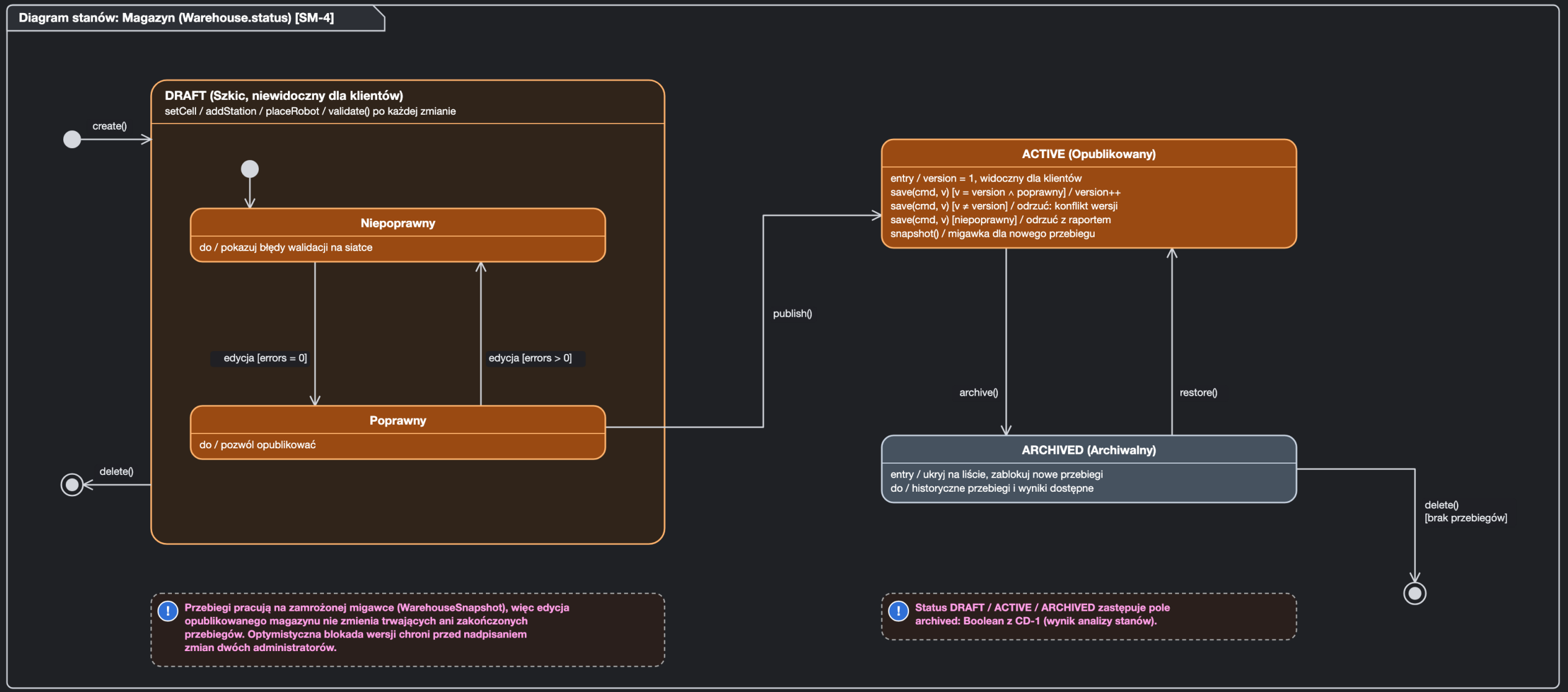
1 pole na tick; pole zajęte → czekaj 1 tick; plan nieaktualny → release() i planuj od nowa

Oczekiwanie

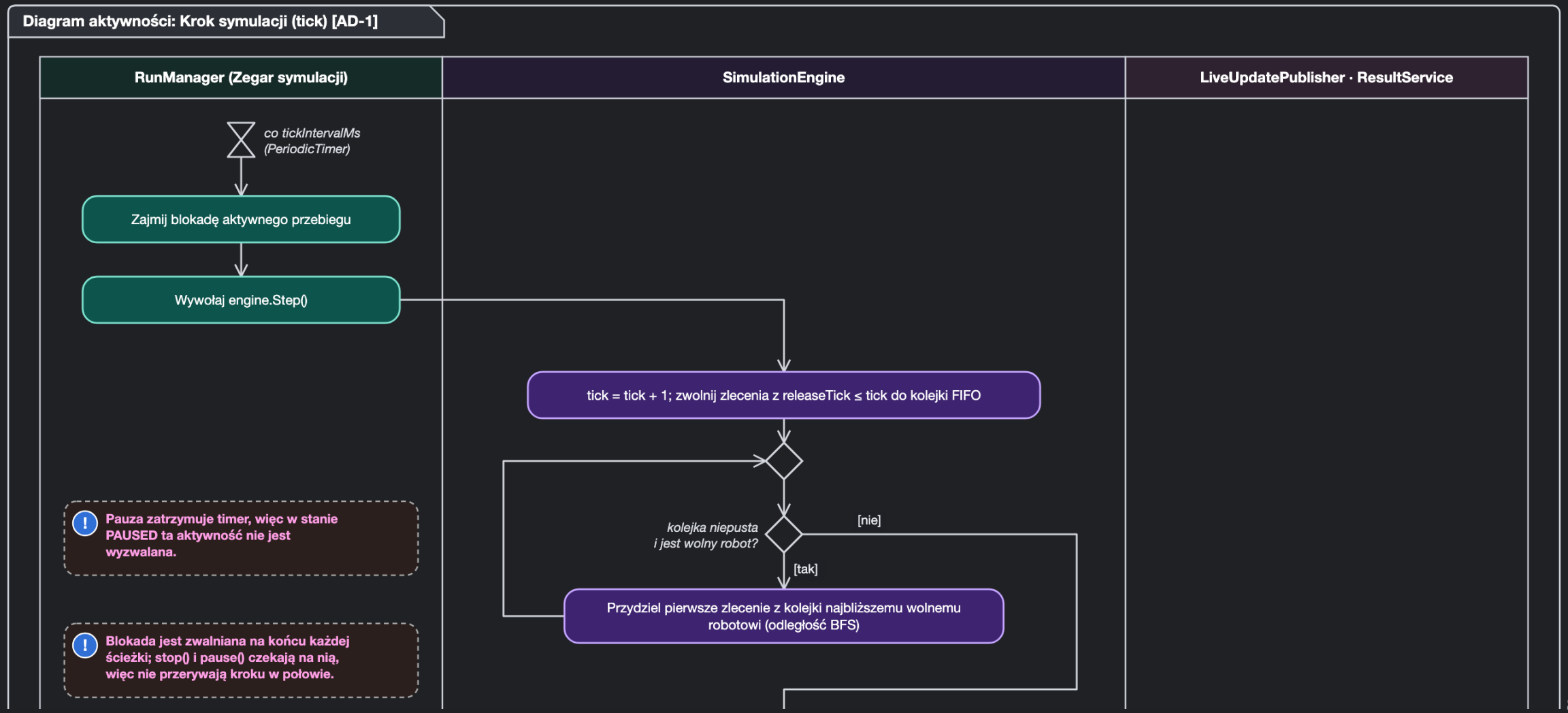
planFailures++, ponowne planowanie po 1 ticku; po yieldAfter tickach zjedź z drogi

Dwa wyjścia

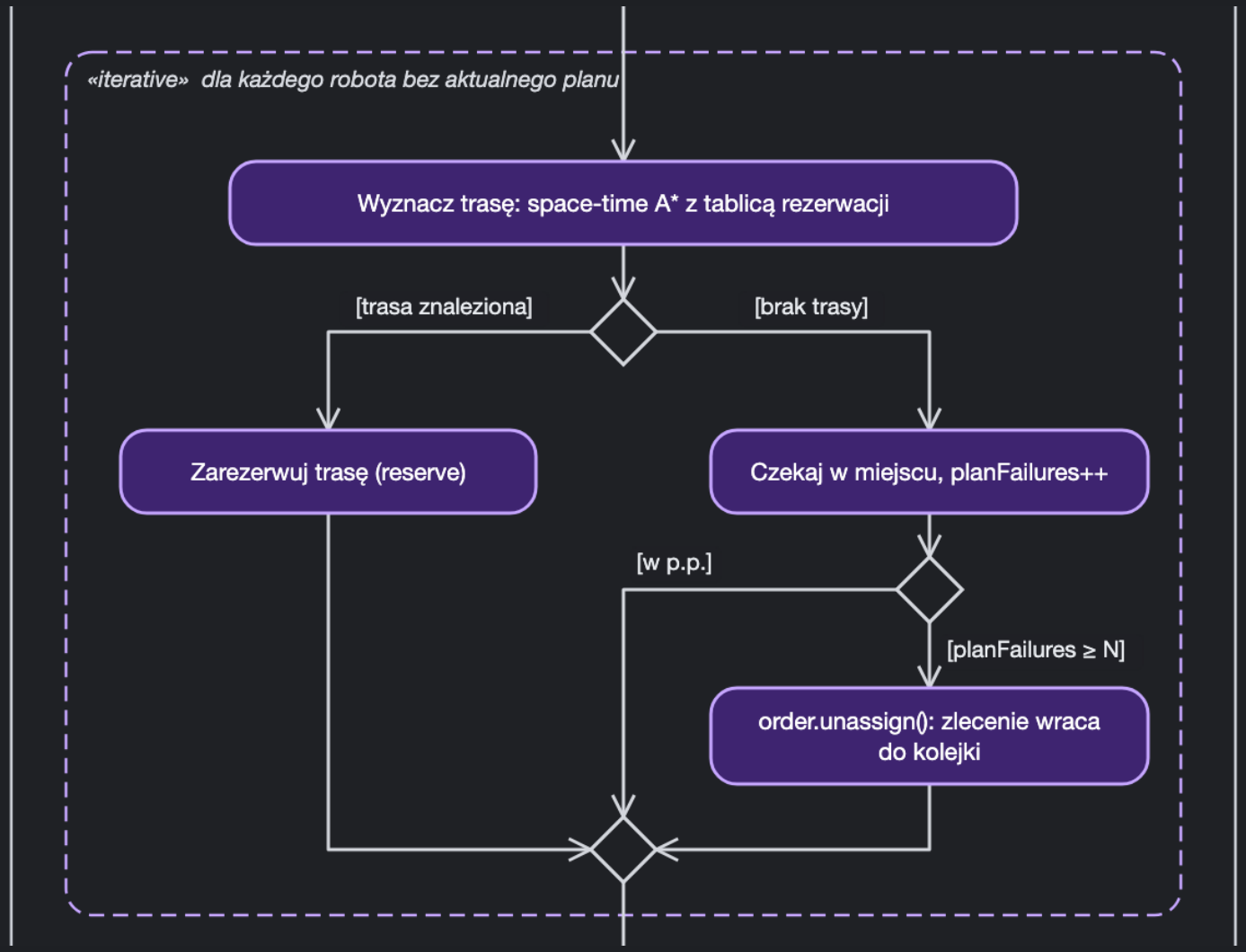
u celu [pozycja = cel] albo rezygnacja [planFailures ≥ N] (tylko z TO_PICKUP)


Diagram stanów: Magazyn (Warehouse.status) [SM-4]
DRAFT (Szkic, niewidoczny dla klientów)
setCell / addStation / placeRobot / validate() po każdej zmianie
create()
Niepoprawny
do / pokazuj błędy walidacji na siatce
edycja [errors = 0]
edycja [errors > 0]
Poprawny
do / pozwól opublikować
delete()
publish()
ACTIVE (Opublikowany)
entry / version = 1, widoczny dla klientów
save(cmd, v) [v = version ∧ poprawny] / version++
save(cmd, v) [v ≠ version] / odrzuć: konflikt wersji
save(cmd, v) [niepoprawny] / odrzuć z raportem
snapshot() / migawka dla nowego przebiegu
archive()
restore()
ARCHIVED (Archiwalny)
entry / ukryj na liście, zablokuj nowe przebiegi
do / historyczne przebiegi i wyniki dostępne
delete()
[brak przebiegów]
Przebiegi pracują na zamrożonej migawce (WarehouseSnapshot), więc edycja opublikowanego magazynu nie zmienia trwających ani zakończonych przebiegów. Optymistyczna blokada wersji chroni przed nadpisaniem zmian dwóch administratorów.
Status DRAFT / ACTIVE / ARCHIVED zastępuje pole archived: Boolean z CD-1 (wynik analizy stanów).

Krok symulacji: zegar, blokada, zwolnienie i przydział zleceń



Krok symulacji: planowanie tras

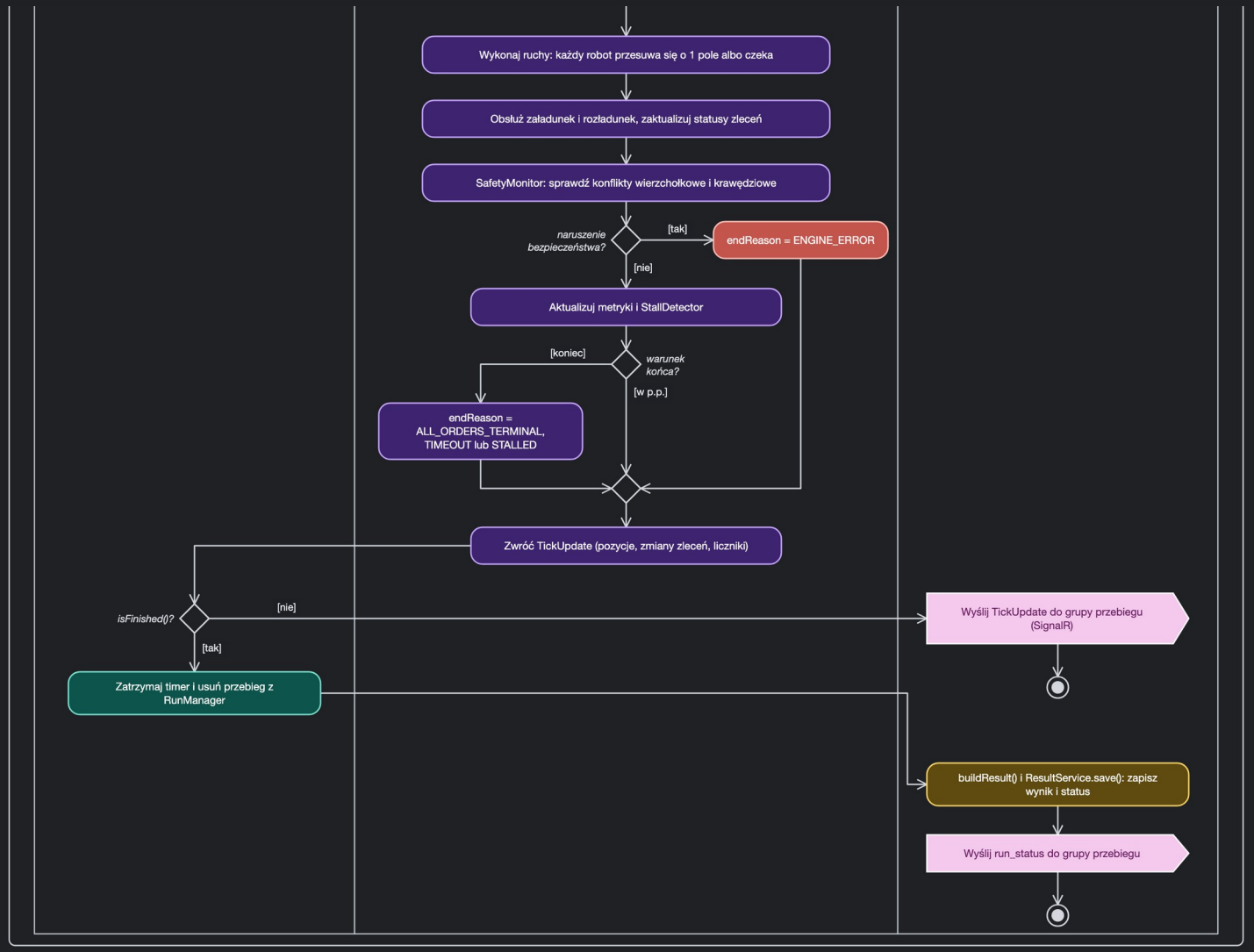


1. **Dla każdego robota bez planu**
   space-time A* z tablicą rezerwacji; trasa znaleziona → reserve()

2. **Brak trasy**
   robot czeka w miejscu, planFailures++; planowanie ponawiane w następnym ticku

3. **planFailures ≥ N**
   order.unassign(): zlecenie wraca do kolejki, robot przechodzi do RETURNING

Krok symulacji: ruch, bezpieczeństwo, koniec i publikacja

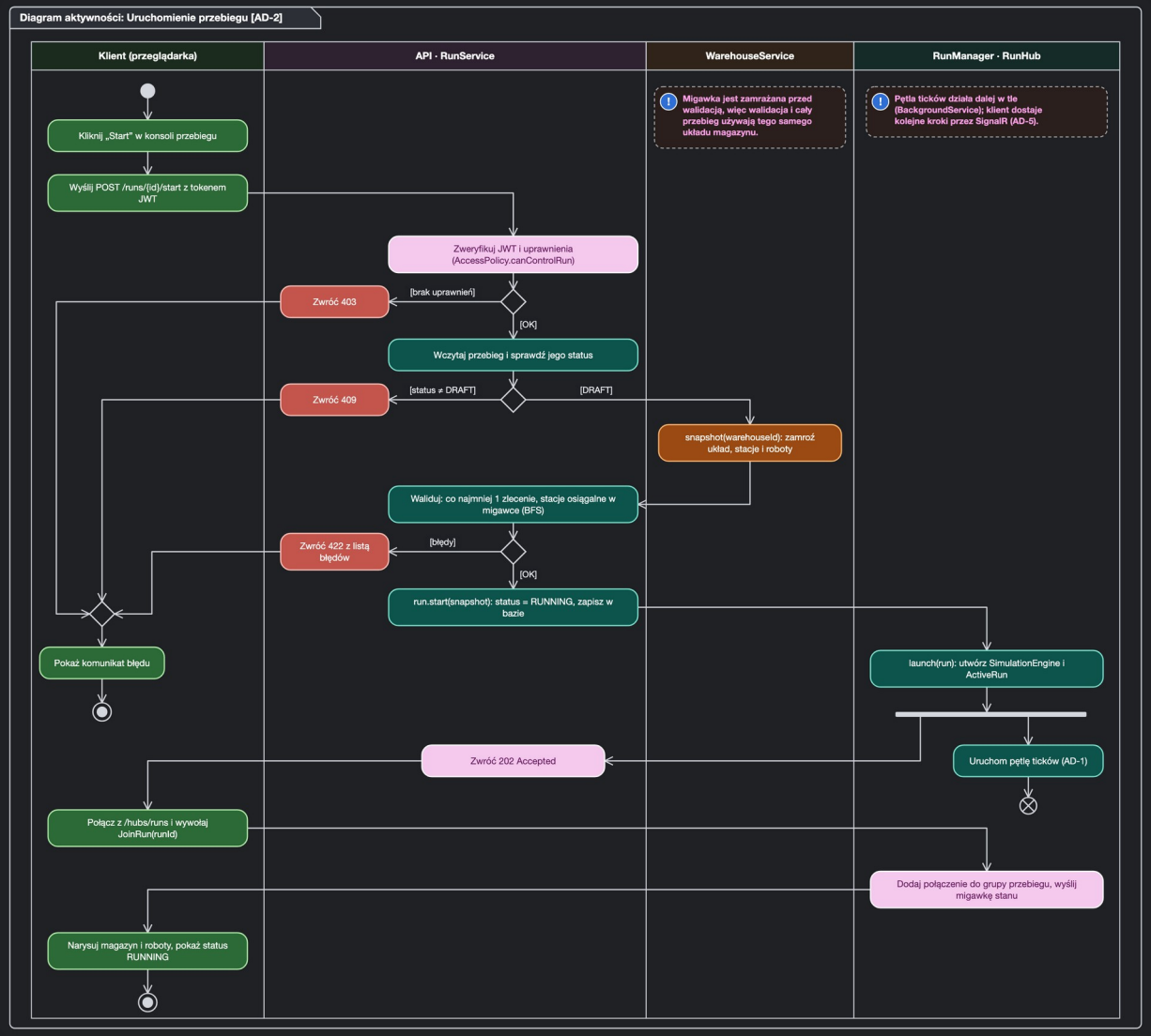


1. **Ruch i obsługa stacji**
   1 pole na tick albo czekanie; załadunek, rozładunek, statusy zleceń

2. **SafetyMonitor**
   konflikt wierzchołkowy lub krawędziowy → ENGINE_ERROR, przebieg FAILED

3. **Warunek końca**
   ALL_ORDERS_TERMINAL, TIMEOUT lub STALLED (StallDetector)

4. **Publikacja**
   TickUpdate przez SignalR; po końcu zapis wyniku i run_status; wynik deterministyczny



AD-2 · URURCHOMIENIE PRZEBIEGU

Trzy bramki, migawka, rozwidlenie

1. **Bramki**
   403 brak uprawnień · 409 status ≠ DRAFT · 422 błędy walidacji; przebieg zostaje szkicem

2. **Migawka przed walidacją**
   snapshot(warehouseId) zamraża układ, stacje i roboty dla całego przebiegu

3. **Rozwidlenie**
   pętla ticków w tle (AD-1) równolegle z odpowiedzią 202 dla klienta

4. **Podgląd**
   JoinRun(runId) w hubie SignalR, migawka stanu, status RUNNING na ekranie

WNIOSKI

Co analiza stanów zmieniła w modelu klas

Warehouse	archived: Boolean → status DRAFT / ACTIVE / ARCHIVED; publish(), archive(), restore()
TransportOrder	nowa operacja unassign(): zlecenie wraca do kolejki po N nieudanych planach
User	stan Dezaktywowane, reactivate(); warunek „nie ostatni administrator”
SimulationRun	stan INTERRUPTED (SERVER_RESTART); RunManager.markInterruptedOnStartup()
RobotAgent	wspólna podmaszyna Ruch; planFailures, yieldAfter; RETURNING → TO_PICKUP bez powrotu do bazy

Zaktualizowane diagramy klas i historyjki użytkownika zaprezentujemy na Checkpoincie 4.